Xv6 Virtual Memory

Student ID：71122135 name：Pengyu Xie

2024 年 5 月 27 日

1 Object

1.1 Null-pointer Dereference

The first thing you might do is create a program that dereferences a null pointer. It is simple! See if you can do it.

1.2 Read-only Code

1.2.1 adding two system calls

In this portion of the xv6 project, you'll change the protection bits of parts of the page table to be read-only, thus preventing such over-writes, and also be able to change them back.

To do this, you'll be adding two system calls: `int mprotect(void *addr, int len)` and `int munprotect(void *addr, int len)`.

Calling mprotect() should change the protection bits of the page range starting at addr and of len pages to be read only. Thus, the program could still read the pages in this range after mprotect() finishes, but a write to this region should cause a trap (and thus kill the process). The munprotect() call does the opposite: sets the region back to both readable and writeable.

There are some failure cases to consider: if addr is not page aligned, or addr points to a region that is not currently a part of the address space, or len is less than or equal to zero, return -1 and do not change anything. Otherwise, return 0 upon success.

1.2.2 be inherited on fork()

Also required: the page protections should be inherited on fork(). Thus, if a process has mprotected some of its pages, when the process calls fork, the OS should copy those protections to the child process.

2 Experimental Procedure

2.1 Null-pointer Dereference

Modify the user program compilation section in the Makefile to set the program entry point to 0x1000.

```
_%: %.o $(ULIB)
    $(LD) $(LDFLAGS) -N -e main -Ttext 0x1000 -o $@ $^
    $(OBJDUMP) -S $@ > $*.asm
    $(OBJDUMP) -t $@ | sed '1,/SYMBOL TABLE/d; s/ \.* / /; /^$$/d' >
        $*.sym

_forktest: forktest.o $(ULIB)
    # forktest has less library code linked in - needs to be small
    # in order to be able to max out the proc table.
    $(LD) $(LDFLAGS) -N -e main -Ttext 0x1000 -o _forktest forktest.
       o ulib.o usys.o
    $(OBJDUMP) -S _forktest > forktest.asm
```

2.2 Read-only Code

2.2.1 adding two system calls

(1) • The function `int mprotect(void *addr, int len)` sets the page table entries corresponding to the range `(addr, addr + len * PGSIZE)` as non-writable.

• The function `int munprotect(void *addr, int len)` sets the page table entries corresponding to the range `(addr, addr + len * PGSIZE)` as writable.

Add the following two functions, `mprotect` and `munprotect`, in the `vm.c` file:

```
int mprotect(void* addr, int len) {
    struct proc* proc = myproc();

    if(len <= 0 || (int)addr + len * PGSIZE > proc->sz) {
         cprintf("\nbad addr!\n");
         return -1;
    }

    if((int)addr % PGSIZE != 0) {
         cprintf("\nnot aligned!\n");
        return -1;
    }

    pte_t* pte;
    for(int i = (int)addr; i < (int)addr + PGSIZE * len; i +=
       PGSIZE) {
        pte = walkpgdir(proc->pgdir, (void*)i, 0);

        if(pte && *pte & PTE_U && *pte & PTE_P) {
```

```
19              *pte &= ~PTE_W;
20          } else {
21              return -1;
22          }
23      }
24
25      lcr3(V2P(proc->pgdir));
26      return 0;
27  }
```

```
 1  int munprotect(void* addr, int len) {
 2      struct proc* proc = myproc();
 3
 4      if(len <= 0 || (int)addr + len * PGSIZE > proc->sz) {
 5          cprintf("\nbad addr!\n");
 6          return -1;
 7      }
 8
 9      if((int)addr % PGSIZE != 0) {
10          cprintf("\nnot aligned!\n");
11          return -1;
12      }
13
14      pte_t* pte;
15      for(int i = (int)addr; i < (int)addr + PGSIZE * len; i +=
            PGSIZE) {
16          pte = walkpgdir(proc->pgdir, (void*)i, 0);
17
18          if(pte && *pte & PTE_U && *pte & PTE_P) {
19              *pte |= PTE_W;
20          } else {
21              return -1;
22          }
23      }
24
25      lcr3(V2P(proc->pgdir));
26      return 0;
27  }
```

(2) Register the system call number for this function in syscall.h, setting it as 22、23. Add the following statement:

```
#define SYS_mprotect 22
#define SYS_munprotect 23
```

(3) Add this system call in `syscall.c`. Add the following statements:

```
...
extern int sys_uptime(void);
extern int sys_mprotect(void);
extern int sys_munprotect(void);

static int (*syscalls[])(void) = {
...
[SYS_close]   sys_close,
[SYS_mprotect] sys_mprotect,
[SYS_munprotect] sys_munprotect,
};
```

(4) Then add the corresponding trampoline function in `usys.S`. In this file, generate all trampoline functions using the preprocessor. Add the following statement:

```
SYSCALL(mprotect)
SYSCALL(munprotect)
```

(5) Finally, add the declaration of the system call in `user.h`:

```
int mprotect(void*, int);
int munprotect(void*, int);
```

3 result

3.1 Null-pointer Dereference

(1) Write a simple user program to attempt dereferencing a null pointer and verify that it indeed triggers an exception in the modified xv6.

The null_deref. c file is as follows:

```c
#include "types.h"
#include "user.h"

int
main(void)
{
    int *p = 0; // Null pointer
    int x = *p; // Dereference null pointer
    printf(1, "x=%d\n", x);
    exit();
}
```

(2) The parts that need to be modified in the Makefile are as follows:

```
UPROGS=\
...
_wc\
_zombie\
_null_deref\
```

(3) The result is shown in the following figure:

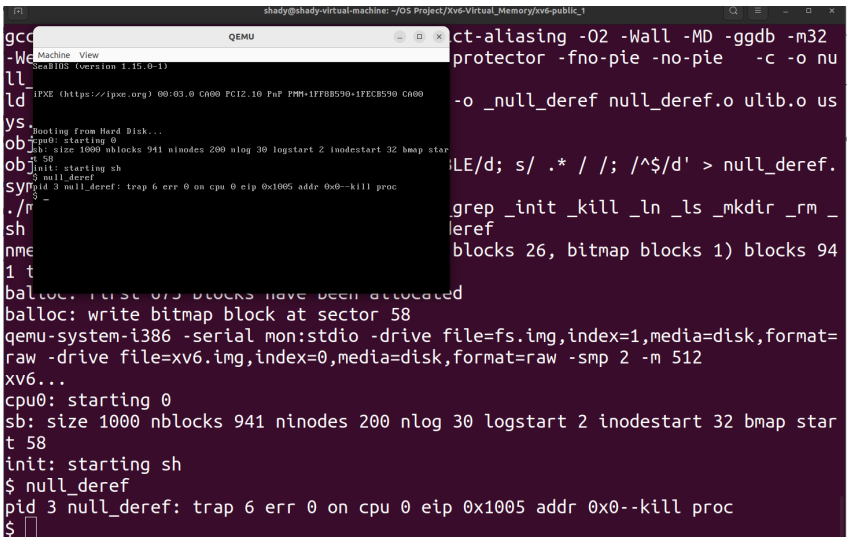

图 1: result for null-pointer dereference

3.2 Read-only Code

3.2.1 adding two system calls

(1) Write test programs `test1.c` and `test2.c` to test the `mprotect` and `munprotect` functions respectively.

The `test1.c` and `test2.c` files are as follows:

```
#include "types.h"
#include "stat.h"
#include "user.h"
#include "fs.h"
#include "mmu.h"

int main(int argc, char *argv[]) {
    char* val = sbrk(0);
    sbrk(PGSIZE);

    *val = 3;
    printf(1, "Start␣at␣%d\n", *val);

    mprotect((void*)val, 1);

    *val = 8;
    printf(1, "Now␣is␣%d\n", *val);

    exit();
}
```

```
#include "types.h"
#include "stat.h"
#include "user.h"
#include "fs.h"
#include "mmu.h"

int main(int argc, char *argv[]) {
    char* val = sbrk(0);
    sbrk(PGSIZE);

    *val = 1;
    printf(1, "Start␣at␣%d\n", *val);

    mprotect((void*)val, 1);
    munprotect((void*)val, 1);
```

```
16
17      *val = 6;
18      printf(1, "Now␣is␣%d\n", *val);
19
20      exit();
21  }
```

(2) The parts that need to be modified in the Makefile are as follows:

```
1  _test1: test1.o $(ULIB)
2          $(LD) $(LDFLAGS) -N -e main -Ttext 0 -o _test1 test1.o $
               (ULIB)
3          $(OBJDUMP) -S _test1 > test1.asm
4          $(OBJDUMP) -t _test1 | sed '1,/SYMBOL␣TABLE/d;␣s/␣.*␣/␣
               /;␣/^$$/d' > test1.sym
5
6  _test2: test2.o $(ULIB)
7          $(LD) $(LDFLAGS) -N -e main -Ttext 0 -o _test2 test2.o $
               (ULIB)
8          $(OBJDUMP) -S _test2 > test2.asm
9          $(OBJDUMP) -t _test2 | sed '1,/SYMBOL␣TABLE/d;␣s/␣.*␣/␣
               /;␣/^$$/d' > test2.sym
```

```
1  UPROGS=\
2          ...
3          _test1\
4          _test2\
```

```
1  EXTRA=\
2          mkfs.c ulib.c user.h cat.c echo.c forktest.c grep.c kill
               .c\
3          ln.c ls.c mkdir.c rm.c stressfs.c usertests.c wc.c
               zombie.c nullptr.c test1.c test2.c test_mprotect.c\
4          printf.c umalloc.c\
5          README dot-bochsrc *.pl toc.* runoff runoff1 runoff.list
               \
6          .gdbinit.tmpl gdbutil\
```

(3) The result is shown in the following figure:

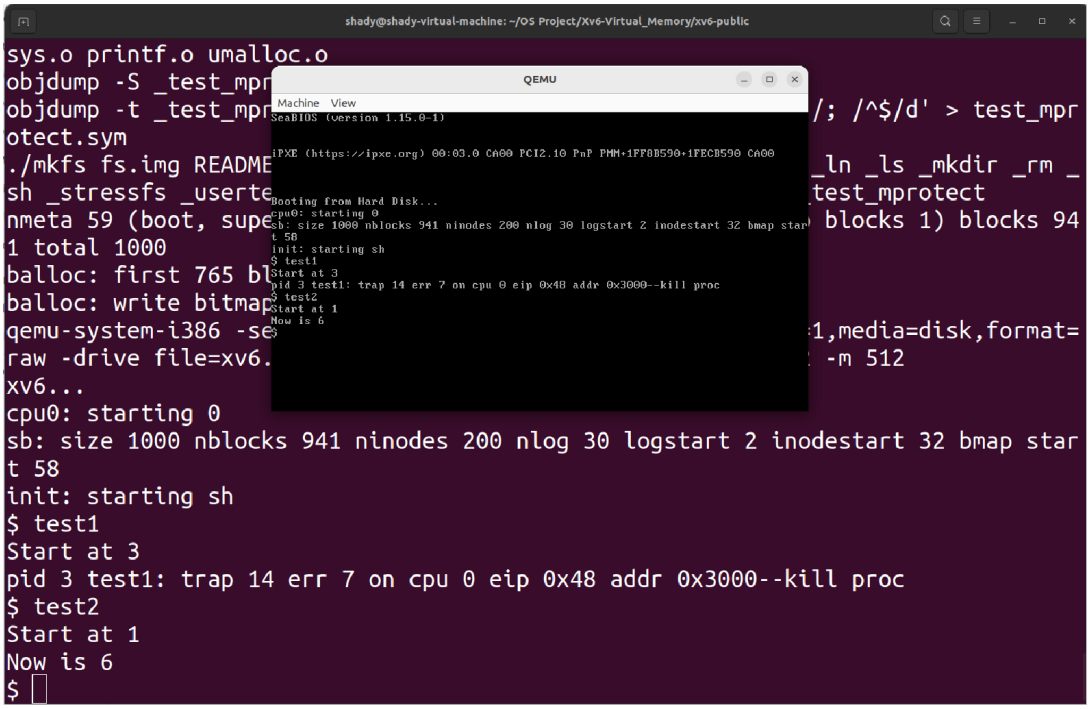

图 2: Page table protection test results

3.2.2 be inherited on fork()

(1) Write test programs `test_mprotect.c` to test the page protections should be inherited on fork().

The `test_mprotect.c` file is as follows:

```c
#include "types.h"
#include "stat.h"
#include "user.h"
#include "fcntl.h"

#define PGSIZE 4096


int main(void) {
    char *addr;
    int pid;

    addr = sbrk(PGSIZE);
    if (addr == (char *)-1) {
        printf(1, "sbrk failed\n");
        exit();
    }
```

```
18
19      if (mprotect(addr, 1) != 0) {
20          printf(1, "mprotect failed\n");
21          exit();
22      }
23
24      pid = fork();
25      if (pid < 0) {
26          printf(1, "fork failed\n");
27          exit();
28      }
29
30      if (pid == 0) {
31          printf(1, "Child process: trying to write to protected
                page\n");
32          *addr = 'X';
33          printf(1, "Child process: write succeeded (this should
                not happen)\n");
34          exit();
35      } else {
36          wait();
37          printf(1, "Parent process: child exited\n");
38      }
39
40      exit();
41  }
```

(2) The parts that need to be modified in the Makefile are as follows:

```
1  _test_mprotect: test_mprotect.o $(ULIB)
2          $(LD) $(LDFLAGS) -N -e main -Ttext 0 -o _test_mprotect
               test_mprotect.o $(ULIB)
3          $(OBJDUMP) -S _test_mprotect > test_mprotect.asm
4          $(OBJDUMP) -t _test_mprotect | sed '1,/SYMBOL TABLE/d; s
               /.*/ /;/^$$/d' > test_mprotect.sym
```

```
1  UPROGS=\
2          ...
3          _test_mprotect\
```

```
1  EXTRA=\
2          mkfs.c ulib.c user.h cat.c echo.c forktest.c grep.c kill
               .c\
3          ln.c ls.c mkdir.c rm.c stressfs.c usertests.c wc.c
               zombie.c nullptr.c test1.c test2.c test_mprotect.c\
4          printf.c umalloc.c\
5          README dot-bochsrc *.pl toc.* runoff runoff1 runoff.list
               \
6          .gdbinit.tmpl gdbutil\
```

(3) The result is shown in the following figure:

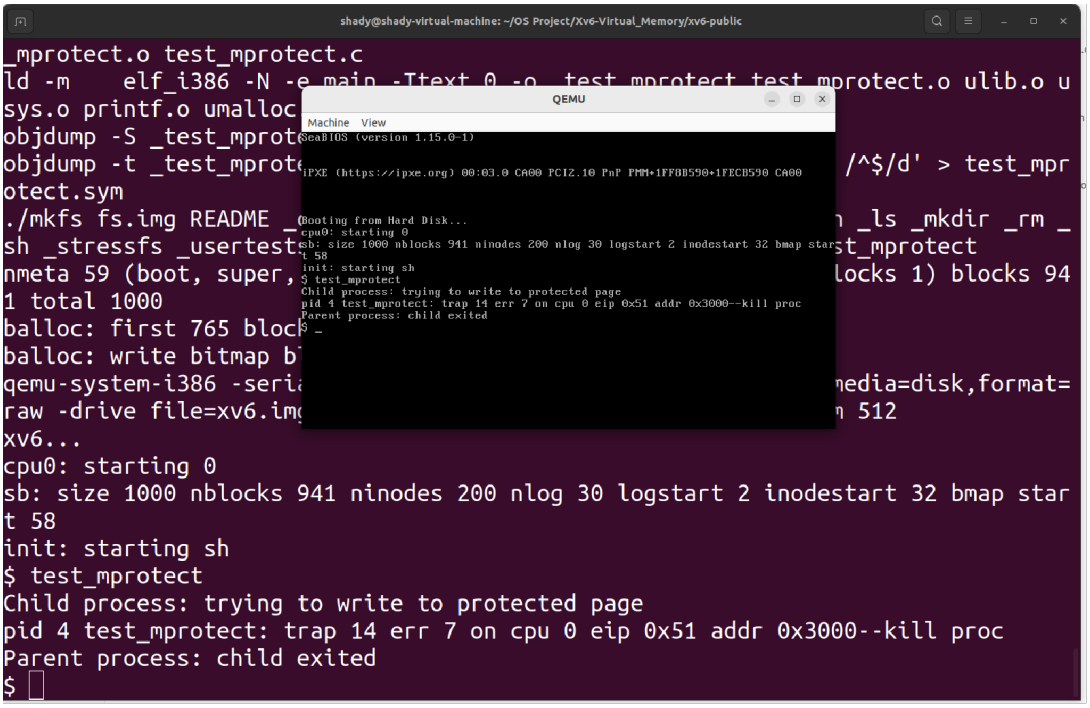

图 3: Inheritance results of the protection function after fork()